

Ingeniería de Datos II

Clase #5

Introducción a Apache Cassandra (CQL)

8 de Abril de 2026

Creación de KeySpace

```
CREATE KEYSPACE test_keyspace  
  WITH replication =  
    {'class':'SimpleStrategy','replication_factor':'1'}  
  AND durable_writes = 'true';
```

- Especificar el nombre del keyspace.
- Especificar los seteos:
 - Setear el factor de replica para # nodes a ser replicados en el cluster. Por ejemplo copiarlos 3 veces sería 3;
 - Especificar la clase de replicación: SimpleStrategy es la más básica, corresponde a un único datacenter;
 - Especificar si queremos escrituras durables: por defecto está en 'true'. 'True' significa que siempre se graba primero el commit log y 'False' significa que se saltea el commit log incrementando el Riesgo de pérdida de datos en caso que el nodo falle, aumentando la velocidad.
- DESCRIBE KEYSPACES: muestra los espacios de claves disponibles.
- USE <nombre del keyspace>: selecciona el espacio de clave.

Creación de KeySpace – cont.

- Un espacio de claves puede contener varias tablas.
- Una tabla contiene un conjunto de filas formadas por pares clave-valor.
- Después de las tablas, trabajamos con filas anchas que constan de una clave primaria y un gran número de columnas.
- Cada fila tiene una clave primaria especificada con acceso a datos.
- La clave primaria es una clave compuesta, formada por la clave de partición, que determina en qué nodos se almacenarán nuestros datos, y el número de columnas de agrupación para ordenar los datos y el orden de almacenamiento en disco.

Creación de Tablas

```
CREATE TABLE employee_by_id (id int PRIMARY KEY, name text, position text);
```

```
CREATE TABLE employee_by_car_make (car_make text, id int, car_model text,  
PRIMARY KEY(car_make, id));
```

```
CREATE TABLE employee_by_car_make_sorted (car_make text, age int, id int,  
name text, PRIMARY KEY(car_make, age, id));
```

```
CREATE TABLE employee_by_car_make_and_model (car_make text,  
car_model text, id int, name text, PRIMARY KEY((car_make, car_model), id));
```

Creación de Tablas – cont.

```
INSERT INTO employee_by_id (id, name, position) VALUES (1, 'John',  
'Manager');
```

```
DROP TABLE employee_by_car_make ;
```

```
DESCRIBE TABLES;
```

Insertando datos en las tablas

```
INSERT INTO employee_by_id (id, name, position) VALUES (2, 'Bob', 'CEO') ;
```

```
INSERT INTO employee_by_car_make (car_make, id, car_model) VALUES  
( 'BMW', 1, 'Sports Car' ) ;
```

```
INSERT INTO employee_by_car_make (car_make, id, car_model) VALUES  
( 'AUDI', 2, 'Truck' ) ;
```

```
INSERT INTO employee_by_car_make (car_make, id, car_model) VALUES  
( 'AUDI', 5, 'Hatchback' ) ;
```

```
INSERT INTO employee_by_car_make (car_make, id, car_model) VALUES  
( 'BMW', 4, 'Sports Car' ) ;
```

Insertando datos en las tablas – cont.

```
INSERT INTO employee_by_car_make_and_model (car_make, car_model, id, name) VALUES ('BMW','HATCHBACK', 1, 'John');
```

```
INSERT INTO employee_by_car_make_and_model (car_make, car_model, id, name) VALUES ('AUDI', 'Truck', 8, 'FRANK');
```

```
INSERT INTO employee_by_car_make_and_model (car_make, car_model, id, name) VALUES ('BMX', 'Truck', 7, 'AMY');
```

```
INSERT INTO employee_by_car_make_and_model (car_make, car_model, id, name) VALUES ('AUDI', 'Sports Car', 4, 'TIM');
```

```
INSERT INTO employee_by_car_make_and_model (car_make, car_model, id, name) VALUES ('AUDI', 'Sports Car', 5, 'JIM');
```

```
INSERT INTO employee_by_car_make_and_model (car_make, car_model, id, name) VALUES ('AUDI', 'Sports Car', 6, 'NICK');
```

Insertando datos en las tablas – cont.

```
INSERT INTO employee_by_car_make (car_make, id, car_model) VALUES  
( 'AUDI', 8, 'Sports Car' ) ;
```

```
INSERT INTO employee_by_car_make (car_make, id, car_model) VALUES  
( 'MERC', 3, 'Saloon' ) ;
```

```
INSERT INTO employee_by_car_make (car_make, id, car_model) VALUES  
( 'MERC', 6, 'Saloon' ) ;
```


Insertando datos en las tablas – cont.

Esto significa que la columna 'name' será null

```
INSERT INTO employee_by_car_make_and_model (car_make, car_model, id)
VALUES ('BMW', 'HATCHBACK', 3);
```

Seleccionando registros de las tablas

```
SELECT * FROM employee_by_car_make WHERE car_make='BMW' ORDER BY id;
```

car_make | car_model | id | name

-----+-----+----+-----

BMW | HATCHBACK | 1 | Bob

BMW | HATCHBACK | 2 | null

BMW | HATCHBACK | 3 | null

```
SELECT * FROM employee_by_car_make_and_model WHERE car_make= 'BMW' AND  
car_model='HATCHBACK' ;
```

car_make | id | car_model

-----+----+-----

BMX | 1 | Sports Car

BMX | 4 | Sports Car

MERC | 3 | Saloon

MERC | 6 | Saloon

AUDI | 2 | Truck

AUDI | 5 | Hatchback

AUDI | 8 | Sports Car

Seleccionando registros de las tablas

Writetime: encontrar cuándo se han insertado los ítems en la tabla

```
SELECT car_make, car_model, writetime(car_model) FROM  
employee_by_car_make ;
```

```
car_make | car_model | writetime(car_model)
```

```
-----+-----+-----
```

```
BMX | Sports Car | 1602018053307000
```

```
BMX | Sports Car | 1602018411020000
```

```
MERC | Saloon | 1602019109699000
```

```
MERC | Saloon | 1602019120387000
```

```
AUDI | Truck | 1602018096459000
```

```
AUDI | Hatchback | 1602018119356000
```

```
AUDI | Sports Car | 1602019087500000
```

Modificar o actualizar valores de una tabla

```
UPDATE employee_by_car_make USING TTL 60 SET car_model='TRUCK'  
WHERE car_make='BMW' AND id=2 ;
```

car_make | id | car_model

-----+-----+-----

BMW | 1 | Sports Car

BMW | 2 | TRUCK

BMW | 4 | Sports Car

MERC | 3 | Saloon

AUDI | 2 | Truck

AUDI | 5 | Hatchback

AUDI | 8 | Sports Car

Agregando una columna a una tabla

```
SELECT * FROM employee_by_id ;
```

id	name	position
----	------	----------

1	John	Manager
2	Bob	CEO

```
ALTER TABLE employee_by_id ADD phone set<text>;
```

```
SELECT * FROM employee_by_id ;
```

id	name	phone	position
----	------	-------	----------

1	John	null	Manager
2	Bob	null	CEO

Actualización alterando valores existentes en tabla usando variables

```
UPDATE employee_by_id SET phone = {'343','565'} WHERE id =1 ;  
SELECT * FROM employee_by_id ;
```

id	name	phone	position
1	John	{'343', '565'}	Manager
2	Bob	null	CEO

Agregando variables

```
UPDATE employee_by_id SET phone = phone + {'555'} WHERE id =1 ;  
SELECT * FROM employee_by_id ;
```

id	name	phone	position
1	John	{'343', '555', '565'}	Manager
2	Bob	null	CEO

Sustrayendo variables

```
UPDATE employee_by_id SET phone = phone - {'565'} WHERE id =1 ;
```

id	name	phone	position
1	John	{'343', '555'}	Manager
2	Bob	null	CEO

ORDER BY solo es soportado en las columnas clusterizadas de la Primary Key

ALLOW FILTERING permite realizar esto pero podría ser perjudicial en términos de performance dado que los datos podrían ser accedidos a través de un gran número de tablas en multiples clusters.

```
SELECT * FROM employee_by_id WHERE name='John' ALLOW FILTERING ;
```

id	name	phone	position
1	John	{'343', '555'}	Manager

Creación de índices

```
CREATE INDEX ON employee_by_id (name);
```

```
SELECT * FROM employee_by_id WHERE name='John' ;
```

id	name	phone	position
----	------	-------	----------

id	name	phone	position
----	------	-------	----------

1	John	{'343', '555'}	Manager
---	------	----------------	---------

Creación de tabla con UUID

```
CREATE TABLE employee_by_uuid (id uuid PRIMARY KEY, first_name text, last_name text);
```

--Insertando para generar uuid

```
INSERT INTO employee_by_uuid (id, first_name, last_name) VALUES (uuid(), 'tom', 'dunne');
INSERT INTO employee_by_uuid (id, first_name, last_name) VALUES (uuid(), 'som', 'dunne');
INSERT INTO employee_by_uuid (id, first_name, last_name) VALUES (uuid(), 'tim', 'smith');
INSERT INTO employee_by_uuid (id, first_name, last_name) VALUES (uuid(), 'bob', 'hanson');
```

```
SELECT * FROM employee_by_uuid;
```

id	first_name	last_name
627a78bc-8fb6-4a4e-9673-9071b6017029	bob	hanson
c2283d7c-ad8a-4e16-8e7a-da23e331c4ee	tim	smith
6394aff2-b0a6-4b36-9519-e0643e6d38a1	tom	dunne
acac2314-4a3f-4a21-ae5a-523f0d61d29f	som	dunne

Creación de tabla con TIMEUUID

```
CREATE TABLE employee_by_timeuuid (id timeuuid PRIMARY KEY, first_name text, last_name text);
```

--Insertando datos

```
INSERT INTO employee_by_timeuuid (id, first_name, last_name) VALUES (now(), 'tim', 'jones') ;
```

```
INSERT INTO employee_by_timeuuid (id, first_name, last_name) VALUES (now(), 'ally', 'smith') ;
```

```
INSERT INTO employee_by_timeuuid (id, first_name, last_name) VALUES (now(), 'kate', 'smith') ;
```

```
SELECT * FROM employee_by_timeuuid;
```

id	first_name	last_name
bd6c28c0-32a9-11f1-893f-4fd0b6ce4ebc	ally	smith
b9a70a70-32a9-11f1-893f-4fd0b6ce4ebc	ally	smith
b9a66e30-32a9-11f1-893f-4fd0b6ce4ebc	tim	jones

Trabajando con contadores

```
CREATE TABLE purchases_by_customer_id (id uuid PRIMARY KEY, purchase counter) ;
```

--Actualización

```
UPDATE purchases_by_customer_id SET purchase = purchase + 1 WHERE id=uuid();
```

```
UPDATE purchases_by_customer_id SET purchase = purchase + 1 WHERE id=uuid();
```

```
UPDATE purchases_by_customer_id SET purchase = purchase + 1 WHERE id=uuid();
```

```
SELECT * FROM purchases_by_customer_id ;
```

id	purchase
-----+-----	
bca1955a-91de-4297-bdfc-c57072bf5897	1
d968b6f2-92fd-4b62-8d3e-772aa8cce416	1
20883123-4dc8-4321-8a53-dc325ab0636b	1

Sumando 1 al contador via update (con id)

```
UPDATE purchases_by_customer_id SET purchase = purchase + 1 where id =  
462c0d3f-c61b-4e4d-9a09-ecac1b3e1ad3;
```

```
SELECT * FROM purchases_by_customer_id ;
```

id	purchase
462c0d3f-c61b-4e4d-9a09-ecac1b3e1ad3	2
bca1955a-91de-4297-bdfc-c57072bf5897	1
6fafc3bf-d5db-4e34-88ca-f64b46e11d7f	1
4cfe057a-19de-4263-a5a6-0d571a21b9cb	1
d968b6f2-92fd-4b62-8d3e-772aa8cce416	1
20883123-4dc8-4321-8a53-dc325ab0636b	1



Practiquemos un poco ...